Rapport

October 15, 2026

1 1. Introduction

La séparation de sources sonores à partir d'un signal audio monophonique est un problème majeur en traitement du signal. Dans ce projet, nous nous intéressons à la décomposition d'un signal audio en ses différentes composantes à partir de son spectrogramme.

Après avoir implémenté *from scratch* la transformée de Fourier à court terme (STFT), nous appliquons des méthodes de **Factorisation en Matrices Non-négatives (NMF)** pour modéliser le spectrogramme. Trois fonctions de coût sont étudiées : la distance euclidienne, la divergence de Kullback-Leibler (KL) et la divergence d'Itakura-Saito (IS).

Pour le rapport, nous utiliserons le fichier piano_mix.wav (moodle) comme base pour comparer nos outils car nous considérons celui-ci comme le plus générique, cependant les méthodes sont utilisables sur d'autre fichier.

2 2. Méthode et outils

2.1 2.1 Implémentation de la Short-Time Fourier Transform (STFT) "from scratch"

Ce notebook détaille les étapes nécessaires pour transformer un signal temporel unidimensionnel en une représentation temps-fréquence (spectrogramme). Le pipeline suit le flux standard : Découpage → Fenêtrage → FFT → Post-traitement.

2.1.1 2.1.1 Implémentation de la Short-Time Fourier Transform (STFT) "from scratch"

Ce notebook détaille les étapes nécessaires pour transformer un signal temporel unidimensionnel en une représentation temps-fréquence (spectrogramme). Le pipeline suit le flux standard : Découpage → Fenêtrage → FFT → Post-traitement.

[98]:
```
# import
import numpy as np
from scipy.signal import stft as scipy_stft
from scipy.signal import istft as scipy_istft
import matplotlib.pyplot as plt
from numpy.linalg import inv
from scipy.io.wavfile import read, write
import IPython
```

2.1.2 2.1.2 Paramétrage du Recouvrement (Hop Size)

La première étape consiste à définir comment la fenêtre glisse sur le signal. On calcule le hop (le pas) en fonction de la taille de la fenêtre et du taux de chevauchement souhaité.

[99]:
```
##########################################################################################
#   DEBUT STFT
##########################################################################################

def compute_hop(nperseg, hop_fraction=0.25):
    """
    Calcule la taille du 'hop' (déplacement en échantillons) et renvoie aussi le recouvrement.

    Paramètres
    ----------
    nperseg : int
        Taille de la fenêtre en échantillons (doit être >= 1).
    hop_fraction : float
        Fraction du nperseg correspondant au hop.
        - Doit être strictement entre 0.0 et 1.0 (exclu).
        - hop_fraction = hop / nperseg.
        - Exemple : hop_fraction=0.25 -> hop = 0.25*nperseg -> overlap = 0.75 (75% recouvrement).

    Retour
    ------
    hop : int
        Nombre d'échantillons de déplacement entre trames (>= 1 et <= nperseg).
    overlap : float
        Fraction de recouvrement entre 0 et <1 (overlap = 1 - hop_fraction).
    """
    # validations
    if not isinstance(nperseg, int) or nperseg < 1:
        raise ValueError("nperseg doit être un entier >= 1.")
    if not isinstance(hop_fraction, (int, float)):
        raise ValueError("hop_fraction doit être un nombre (float).")
    hop_fraction = float(hop_fraction)

    if not (0.0 < hop_fraction < 1.0):
        raise ValueError("hop_fraction doit être dans l'intervalle (0.0, 1.0) une fraction quoi. "
                         "Exemples : 0.5 -> 50pourencent overlap (hop = nperseg*0.5), "
                         "0.25 -> 75pourcent overlap (hop = nperseg*0.25).")

    # calcul du hop (arrondi au plus proche entiers)
    hop = int(round(nperseg * hop_fraction))
```

```
    # protections : s'assurer que hop est dans [1, nperseg]
    if hop < 1:
        hop = 1
    if hop > nperseg:
        hop = nperseg

    overlap = 1.0 - (hop / float(nperseg))

    return hop, overlap
```

2.1.3 2.1.3 Segmentation du Signal (Framing)

Cette fonction transforme le signal 1D en une matrice 2D où chaque ligne correspond à une portion temporelle (trame). Un Zero-Padding est appliqué à la fin du signal pour s'assurer que la dernière fenêtre soit complète.

[100]:
```
# Framing
def framing(x, nperseg, hop):
    """
    Découpe le signal 1D `x` en trames (frames) successives.

    Paramètres
    ----------
    x : ndarray 1D
        Signal d'entrée (supposée mono).
    nperseg : int
        Taille de chaque frame en échantillons (>=1).
    hop : int
        Déplacement en échantillons entre le début de deux frames (>=1, <=␣
 ↪nperseg).

    Remarques
    ---------
    - Cette version simplifiée FORCE le padding de la fin du signal (padding␣
 ↪constant à zéro)
      afin que la dernière frame soit complète : on n'accepte plus d'options␣
 ↪pour le padding.

    - Le pad_mode est fixé à 'constant' (zéros).
    - Le comportement garantit au moins une frame si le signal est plus court␣
 ↪que nperseg.
    """

    # validations simples
    if not isinstance(nperseg, int) or nperseg < 1:
        raise ValueError("nperseg doit être un entier >= 1.")
```

```
    if not isinstance(hop, int) or hop < 1:
        raise ValueError("hop doit être un entier >= 1.")
    if hop > nperseg:
        raise ValueError("hop ne peut pas être supérieur à nperseg.")
    if x.ndim != 1:
        raise ValueError("x doit être un tableau 1D (signal mono).")

    N = len(x)
    if N == 0:
        return np.zeros((0, nperseg), dtype=x.dtype), np.zeros((0,), dtype=int)

    # --- calcul du nombre de frames (padding forcé à la fin, pad_mode='constant') ---
    # on autorise la dernière fenêtre à démarrer avant la fin et on complète si nécessaire
    n_frames_calc = int(np.floor((N - nperseg) / float(hop))) + 1
    # garantir au moins 1 frame (cas len(x) < nperseg)
    n_frames = max(1, n_frames_calc)

    # indices de départ
    starts = np.arange(n_frames) * hop

    # padding constant à la fin si nécessaire (pad_mode = 'constant')
    last_needed = starts[-1] + nperseg
    if last_needed > N:
        pad_len = last_needed - N
        x_padded = np.pad(x, (0, pad_len), mode='constant')
    else:
        x_padded = x

    # construire les frames
    frames = np.empty((n_frames, nperseg), dtype=x_padded.dtype)
    for i, s in enumerate(starts):
        frames[i, :] = x_padded[s : s + nperseg]

    return frames, starts
```

2.1.4 2.1.4 Gestion de la Fenêtre Temporelle

Pour éviter les discontinuités aux bords des trames (fuites spectrales), on applique une fenêtre de pondération. Ici, nous utilisons la fenêtre de Hann, idéale car elle respecte la condition COLA (Constant Overlap-Add) pour une reconstruction parfaite ultérieure.

$$w(n) = 0.5 - 0.5\cos\left(\frac{2\pi n}{M-1}\right)$$

[101]:
```
# la forme de notre fenetre qu'on va faire glisser sur le signal
def get_window(window_name="hann", nperseg=1024):
```

```
    """
    Génère une fenêtre temps de longueur `nperseg`.

    Paramètres
    ----------
    window_name : str
        Nom de la fenêtre souhaitée (insensible à la casse).
        Dans notre projet on n'utilise QUE :
            - 'hann' (recommandé pour STFT/ISTFT, respecte COLA avec hop =␣
↪nperseg/2)

        Les autres fenêtres sont supprimées car :
            - 'rect' provoque des fuites spectrales
            - 'hamming', 'blackman', 'bartlett' compliquent la reconstruction
            - 'kaiser' inutile ici et nécessite un paramètre
    nperseg : int
        Longueur de la fenêtre en échantillons (>= 1).

    Retour
    ------
    w : ndarray, shape (nperseg,)
        Vecteur fenêtre.

    Remarques / recommandations
    ---------------------------
    - 'hann' est un excellent compromis pour STFT et ISTFT.
      Avec hop = nperseg/2 → condition COLA vérifiée.
      (somme_k w[n - k*hop] = C)
    """

    # validations d'entrée
    if not isinstance(nperseg, int) or nperseg < 1:
        raise ValueError("nperseg doit être un entier >= 1.")

    # accepter seulement 'hann' qui est suffisant (je pense)
    wn = str(window_name).lower()
    if wn not in ('hann', 'hanning'):
        raise ValueError(
            "Seule la fenêtre 'hann' est supportée dans notre implémentation␣
↪simplifiée."
        )


    if nperseg > 1:
        n = np.arange(nperseg)
        w = 0.5 - 0.5 * np.cos(2.0 * np.pi * n / (nperseg - 1))
    else:
```

```
        w = np.array([1.0])

    return w.astype(float)
```

[102]:
```
def apply_window(frames, window):
    """
    Applique une fenêtre à chaque trame.

    Paramètres
    ----------
    frames : ndarray (n_frames, nperseg)
        Trames du signal
    window : ndarray (nperseg,)
        Vecteur fenêtre

    Retour
    ------
    windowed_frames : ndarray (n_frames, nperseg)
        Trames fenêtrées (element-wise multiplication)

    Exemple
    -------
    >>> frames = np.array([[1, 2, 3], [4, 5, 6]])
    >>> window = np.array([0.5, 1.0, 0.5])
    >>> apply_window(frames, window)
    array([[0.5, 2. , 1.5],
           [2. , 5. , 3. ]])
    """
    # Validation des dimensions
    if frames.ndim != 2:
        raise ValueError(f"frames doit être 2D, shape reçu: {frames.shape}")
    if window.ndim != 1:
        raise ValueError(f"window doit être 1D, shape reçu: {window.shape}")
    if frames.shape[1] != window.shape[0]:
        raise ValueError(
            f"Dimension incompatible: frames.shape[1]={frames.shape[1]} "
            f"mais window.shape[0]={window.shape[0]}"
        )

    # Multiplication élément par élément (broadcasting)
    return frames * window
```

2.1.5 2.1.5 Analyse Spectrale (FFT Réelle)

Nous passons du domaine temporel au domaine fréquentiel. Puisque le signal audio est réel, la sortie de la FFT est symétrique ; nous n'utilisons donc que la moitié des points (rfft) pour optimiser les calculs.

[103]:
```python
def fft_frames(windowed_frames, nfft=None, fs=1.0):
    """
    Applique la FFT réelle (rFFT) à chaque trame fenêtrée.

    Paramètres
    ----------
    windowed_frames : ndarray (n_frames, nperseg)
        Trames fenêtrées (signal réel).
    nfft : int ou None
        Taille de la FFT. Si None, nfft = nperseg.
        Si nfft > nperseg, zero-padding est appliqué.
    fs : float
        Fréquence d'échantillonnage en Hz (pour calculer les fréquences).
        Par défaut 1.0 (fréquences normalisées).

    Retour
    ------
    STFT : ndarray complex (n_frames, n_freq_bins)
        Transformée de Fourier par trame.
        n_freq_bins = nfft//2 + 1 (rFFT pour signaux réels).
    freqs : ndarray (n_freq_bins,)
        Fréquences en Hz correspondant aux bins de fréquence.

    Notes
    -----
    - Utilise np.fft.rfft() qui est optimisé pour les signaux réels.
    - n_freq_bins = nfft//2 + 1 (ex: nfft=1024 → 513 bins de fréquence).
    - La fréquence de Nyquist est fs/2 (incluse si nfft est pair).
    """
    # Validation des entrées
    if windowed_frames.ndim != 2:
        raise ValueError(f"windowed_frames doit être 2D, shape reçu:␣
↪{windowed_frames.shape}")

    n_frames, nperseg = windowed_frames.shape

    # Déterminer nfft
    if nfft is None:
        nfft = nperseg
    elif nfft < nperseg:
        raise ValueError(f"nfft ({nfft}) ne peut pas être < nperseg␣
↪({nperseg})")
    elif nfft <= 0:
        raise ValueError(f"nfft doit être > 0, reçu: {nfft}")

    if fs <= 0:
        raise ValueError(f"fs doit être > 0, reçu: {fs}")
```

```
    # Appliquer la FFT réelle (rfft) sur chaque trame
    # axis=1 : FFT le long des colonnes (pour chaque trame)
    STFT = np.fft.rfft(windowed_frames, n=nfft, axis=1) # supposé 1d

    # np.fft.rfftfreq(nfft, d=1/fs) donne les fréquences pour un rfft
    freqs = np.fft.rfftfreq(nfft, d=1.0/fs)

    return STFT, freqs
```

[104]:
```
def magnitude_or_power(STFT, kind="magnitude"):
    """
    Convertit la STFT complexe en magnitude ou puissance.
    """
    if kind == "magnitude":
        return np.abs(STFT)
    elif kind == "power":
        return np.abs(STFT) ** 2
    else:
        raise ValueError("kind doit être 'magnitude' ou 'power'")
```

2.1.6 2.1.6 Fonction Principale : L'Orchestrateur stft

Cette fonction regroupe toutes les étapes précédentes dans un pipeline unique et cohérent. Elle permet de générer soit une matrice complexe, soit un spectrogramme (magnitude/puissance) avec des options de filtrage fréquentiel.

[105]:
```
def stft(
    x,
    fs,
    nperseg,
    window="hann",
    hop_fraction=0.25,       # la partie de la fenetre qui correspond au hop␣
 ↪cad : hop_fraction = hop / nperseg
    nfft=None,
    output="magnitude",      # "magnitude" | "power" | "complex"
    keep_freq_max=None,      # float Hz or None (pour couper la frequence,␣
 ↪genre afficher jusqu'a f_lim)
    normalize=False,         # bool : normaliser M par sa valeur max
    save_path=None,          # str or None : chemin pour save_matrix()
    visualize=False,         # bool : appeler visualize_spectrogram()
    return_extra=False       # bool : retourne frames, window, STFT en plus
):
    """
    STFT - fonction orchestration (pipeline STFT 'from scratch').

    Cette fonction coordonne les étapes :
```

```
    - calcul du hop à partir de nperseg et hop_fraction
    - framing (découpage du signal en trames)
    - génération de la fenêtre (window)
    - fenêtrage des trames
    - FFT frame-by-frame (avec éventuel zero-padding nfft)
    - conversion en magnitude ou power (ou renvoyer STFT complexe)
    - cropping fréquentiel optionnel (keep_freq_max)
    - visualisation / retour

  Paramètres attendus (résumé) :
    - x : ndarray 1D (signal)
    - fs : int (échantillonnage en Hz)
    - nperseg : int (taille fenêtre en échantillons)
    - window : str or callable (nom de fenêtre, ex "hann")
    - hop_fraction : float dans (0,1) . Exemple 0.25 -> 75% overlap
    - nfft : int ou None. Si None, nfft = max(nperseg, 2048)
    - output : "magnitude", "power" ou "complex"
    - keep_freq_max : fréquence max à conserver (Hz) ou None
    - normalize : si True, M /= M.max()
    - save_path : si fourni, appelle save_matrix(M, freqs, times, meta,␣
↪save_path)
    - visualize : si True, appelle visualize_spectrogram(M, freqs, times,␣
↪f_lim=keep_freq_max_or_default)
    - return_extra : si True, retourne (M, freqs, times, meta, extras_dict)

  Retours :
    - si return_extra == False :
        (M, freqs, times, meta)
    - si return_extra == True :
        (M, freqs, times, meta, {"frames": frames, "window": window_vec,␣
↪"STFT": STFT})
    - Convention de forme :
        M : ndarray shape (n_freq_bins, n_time_frames)  # recommandé freq x␣
↪time
        freqs : ndarray length n_freq_bins (Hz)
        times : ndarray length n_time_frames (s)
        meta : dict (fs, nperseg, hop, nfft, window_name, output,␣
↪keep_freq_max, normalize)

  Remarques d'implémentation :
    - Cette fonction suppose l'existence des helpers :
      compute_hop(nperseg, hop_fraction) -> int hop
      framing(x, nperseg, hop) -> frames (n_frames, nperseg)
      get_window(window_name, nperseg) -> window_vec (nperseg,)
      apply_window(frames, window_vec) -> windowed_frames
      fft_frames(windowed_frames, nfft) -> STFT (n_frames, n_freqs), freqs␣
↪(n_freqs,)
```

```
      magnitude_or_power(STFT, kind="magnitude"|"power") -> M (n_freqs,␣
↪n_frames)
      save_matrix(M, freqs, times, meta, path)
      visualize_spectrogram(M, freqs, times, f_lim)
    - times are computed as the center times of each frame:
        frame_center_sample = first_frame_start + frame_index * hop + nperseg/
↪2
        time = frame_center_sample / fs
    - STFT returned by fft_frames is expected shape (n_frames, n_freqs) (time␣
↪major).
    - stft() converts that to M with shape (n_freqs, n_frames) for convention␣
↪freq x time.

  """
  # ---------------------------
  # 1) Validation & defaults
  # ---------------------------


  if x is None:
      raise ValueError("Signal x is None")
  if x.ndim != 1:
      raise ValueError("Signal x must be 1D (mono). Convert to mono before␣
↪calling stft().")
  if nperseg <= 0:
      raise ValueError("nperseg must be a positive integer.")
  if not (0.0 < hop_fraction < 1.0):
      raise ValueError("hop_fraction must be in (0,1). Example: 0.25 for 75%␣
↪overlap.")

  if nfft is None:
      nfft = nperseg

  # ---------------------------
  # 2) compute hop
  # ---------------------------
  hop, overlap = compute_hop(nperseg, hop_fraction)  # retourne (hop, overlap)
  # On garde seulement hop pour le reste, overlap sera ajouté dans meta

  # ---------------------------
  # 3) framing
  # ---------------------------
  frames, starts = framing(x, nperseg=nperseg, hop=hop)  # shape (n_frames,␣
↪nperseg)
  n_frames = frames.shape[0]
```

```
    # ---------------------------
    # 4) window generation & apply
    # ---------------------------
    window_vec = get_window(window, nperseg)      # shape (nperseg,)
    windowed = apply_window(frames, window_vec)  # shape (n_frames, nperseg)

    # ---------------------------
    # 5) FFT par frame
    # ---------------------------
    STFT, freqs = fft_frames(windowed, nfft=nfft, fs=fs)
    # STFT expected shape: (n_frames, n_freqs), freqs shape (n_freqs,)

    # ---------------------------
    # 6) magnitude/power/keep complex
    # ---------------------------
    if output == "complex":
        M_complex = STFT.T
        M = None
    elif output in ("magnitude", "power"):
        M_tmp = magnitude_or_power(STFT, kind=output)
        # Ensure convention freq x time
        if M_tmp.shape[0] == STFT.shape[0]:
            # If helper returned time-major (n_frames, n_freqs), transpose time␣
↪et freq sont inversées en gros
            M = M_tmp.T
        else:
            M = M_tmp
    else:
        raise ValueError("output must be one of 'magnitude', 'power',␣
↪'complex'")

    # ---------------------------
    # 7) time vector (frame centers)
    # ---------------------------
    # first frame start sample est starts[0] (pas forcément 0)
    # frame_centers_samples = (np.arange(n_frames) * hop) + (nperseg / 2.0)
    frame_centers_samples = starts + (nperseg / 2.0)
    times = frame_centers_samples / float(fs)   # seconds

    # ---------------------------
    # 8) cropping frequency dimension (optionell)
    # ---------------------------
    if keep_freq_max is not None:
        # freqs is length n_freqs (Hz)
        freq_mask = freqs <= float(keep_freq_max)
        freqs_sel = freqs[freq_mask]
```

```
        if output == "complex":
            M_complex = M_complex[freq_mask, :] if M_complex is not None else␣
↪None
        else:
            M = M[freq_mask, :] if M is not None else None
    else:
        freqs_sel = freqs



    # ----------------------------
    #  metadata assembly
    # ----------------------------
    meta = {
        "fs": int(fs),
        "nperseg": int(nperseg),
        "hop": int(hop),
        "hop_fraction": float(hop_fraction),
        "overlap": float(overlap),
        "nfft": int(nfft),
        "window": str(window),
        "output": str(output),
        "keep_freq_max": (float(keep_freq_max) if keep_freq_max is not None␣
↪else None),
        "normalize": bool(normalize),
        "starts": starts.tolist()
    }

    # ----------------------------
    # 12) return values
    # ----------------------------
    if return_extra:
        extras = {
            "frames": frames,
            "window": window_vec,
            "STFT": STFT,
            "starts": starts
        }
        if output == "complex":
            # return complex STFT in the place of M
            return M_complex, freqs_sel, times, meta, extras
        return M, freqs_sel, times, meta, extras

    # no extras
    if output == "complex":
        return M_complex, freqs_sel, times, meta
    return M, freqs_sel, times, meta
```

[106]:
```
# chargement audio
fs, x = read('piano_mix.wav')
t_x = np.arange(len(x))/fs
```

[107]:
```
nperseg = 2048
hop_fraction = 0.25  # 75% overlap
window = 'hann'
hop = int(nperseg * hop_fraction)
noverlap = nperseg - hop
```

[108]:
```
M, freqs, times, meta = stft(
    x, fs, nperseg,
    window=window,
    hop_fraction=hop_fraction,
    output="complex",
)
```

2.2 2.2 Factorisation en Matrices Non-négatives (NMF)

2.2.1 2.2.1 Initialisation et reproductibilité

Les algorithmes de NMF sont sensibles à l'initialisation. Afin de garantir une comparaison équitable entre les différentes méthodes, nous fixons une graine aléatoire commune (`seed=42`) pour forcer le générateur de nombres aléatoires à produire la même séquence, assurant que les méthodes démarrent avec les mêmes conditions initiales.

[109]:
```
np.random.seed(42)
%matplotlib inline
```

2.2.2 2.2.2 Distance Euclidienne (Frobenius)

La méthode implémentée ici utilise la **distance Euclidienne** (norme de Frobenius) comme fonction de coût. Elle cherche à minimiser l'erreur de reconstruction entre la matrice originale M et son approximation $D \times A$:

$$J(D, A) = \frac{1}{2}\|M - DA\|_F^2$$

2.2.3 Règles de mise à jour (Euclidienne)

Les mises à jour multiplicatives pour cette méthode sont dérivées du gradient de la fonction de coût :

1. **Mise à jour des Activations** (A) :

$$A \leftarrow A \otimes \frac{D^T M}{D^T DA}$$

2. **Mise à jour du Dictionnaire** (D) :

$$D \leftarrow D \otimes \frac{MA^T}{DAA^T}$$

[110]:
```python
def NMF_Euclidienne(M, S, epsilon=0.006, MAX_iter=5000):
    """
    Implémentation de la NMF basée sur la distance Euclidienne.
    M : Matrice d'entrée (K, N)
    S : Nombre de composants (rang de la décomposition)
    """
    np.random.seed(42)
    counter = 0
    beta_divergence = 1

    K, N = np.shape(M)
    cost_function = []

    # Initialisation de W (D) et H (A) matrices
    D = np.abs(np.random.normal(loc=0, scale=2.5, size=(K, S)))
    A = np.abs(np.random.normal(loc=0, scale=2.5, size=(S, N)))

    last_betha = beta_divergence + epsilon

    while abs(last_betha - beta_divergence) >= epsilon and counter <= MAX_iter:
        last_betha = beta_divergence

        # --- Mise à jour de A (Activations) ---
        numerator_A = D.T @ M
        denominator_A = D.T @ D @ A
        # On ajoute 1e-9 pour éviter la division par zéro
        A = A * (numerator_A / (denominator_A + 1e-9))

        # --- Mise à jour de D (Dictionnaire) ---
        numerator_D = M @ A.T
        denominator_D = D @ A @ A.T
        D = D * (numerator_D / (denominator_D + 1e-9))

        # Calcul du coût (Norme de Frobenius)
        beta_divergence = np.linalg.norm(M - D @ A)
        cost_function.append(beta_divergence)

        counter += 1

    return D, A, cost_function
```

2.2.4 2.2.3 Divergence de Kullback–Leibler (KL)

La divergence KL est issue de la théorie de l'information. Elle permet de mesurer l'écart entre un spectrogramme observé V et sa reconstruction WH.

La condition d'arrêt est définie par :

$$|\log(J_{k-1}) - \log(J_k)| < \epsilon$$

2.2.5 Règles de mise à jour (KL)

1. **Mise à jour de matrice H :**

$$H \leftarrow H \otimes \frac{W^T(V \oslash WH)}{W^T\mathbf{1}}$$

2. **Mise à jour du dictionnaire W :**

$$W \leftarrow W \otimes \frac{(V \oslash WH)H^T}{\mathbf{1}H^T}$$

Notations : * $\otimes$: multiplication élément par élément * $\oslash$: division élément par élément * $\mathbf{1}$: matrice de 1 de même taille que V

[111]:
```python
def nmf_kl(M, S, epsilon=0.06, MAX_iter=4000 , beta=1):

    np.random.seed(42)


    counter = 0

    #cost_function = []
    kl_divergence = 1

    K, N = M.shape
    cost_function = []
    # Initialisation of W and H matrices : The initialization is generally␣
↪random
    D = np.abs(np.random.normal(loc=0, scale = 2.5, size=(K,S)))
    A = np.abs(np.random.normal(loc=0, scale = 2.5, size=(S,N)))
    last_kl = kl_divergence + epsilon
    while abs(last_kl - kl_divergence) >= epsilon and counter <= MAX_iter:
        last_kl = kl_divergence
        # Update of A and D

        A *= (D.T @ (M / (D @ A))) / (D.T @ np.ones((K, N)))
        D *= ((M / (D @ A)) @ A.T ) / (np.ones((K, N)) @ A.T)
        #divergence
        last_kl= np.sum(M * np.log(M / (D @ A)) - M + (D@A) )
        cost_function.append( last_kl )
        counter+=1

    return D,A,cost_function
```

2.2.6 2.2.4 Divergence d'Itakura–Saito (IS)

La divergence IS est invariante à l'échelle, ce qui la rend particulièrement adaptée à l'analyse des spectres de puissance audio.

2.2.7 Règles de mise à jour (IS)

$$H \leftarrow H \otimes \frac{W^T(V \oslash (WH)^2)}{W^T(1 \oslash WH)}$$

$$W \leftarrow W \otimes \frac{(V \oslash (WH)^2)H^T}{(1 \oslash WH)H^T}$$

[112]:
```
def nmf_is(V, K, max_iter=1000, tol=1e-5, eps=1e-10):
    F, T = V.shape
    W = np.random.rand(F, K) + eps
    H = np.random.rand(K, T) + eps
    costs = []

    for i in range(max_iter):
        WH = W @ H + eps

        # Mise à jour H
        H *= (W.T @ (V / WH**2)) / (W.T @ (1 / WH) + eps)

        WH = W @ H + eps
        # Mise à jour W
        W *= ((V / WH**2) @ H.T) / ((1 / WH) @ H.T + eps)

        # Normalisation pour éviter la dérive d'échelle
        W /= np.sum(W, axis=0, keepdims=True)

        # Calcul du coût chaque 10 itérations pour optimiser
        if i % 10 == 0:
            cost = np.sum(V / WH - np.log(V / WH + eps) - 1)
            costs.append(cost)
            if len(costs) > 1 and abs(costs[-2] - cost) < tol: break

    return W, H, costs
```

2.2.8 2.2.5 Visualisation et Analyse

2.2.9 Visualisation des Matrices

La figure suivante montre le dictionnaire de fréquences (W), les activations temporelles (H) et le spectrogramme original (V).

[113]:
```
# Exécution Euclidienne (M doit être défini au préalable par la STFT)
We, He, costs_E = NMF_Euclidienne(np.abs(M), S=3)

fig = plt.figure(figsize=(14, 6))
gs = fig.add_gridspec(2, 3, width_ratios=[1, 4, 4], height_ratios=[1, 10])

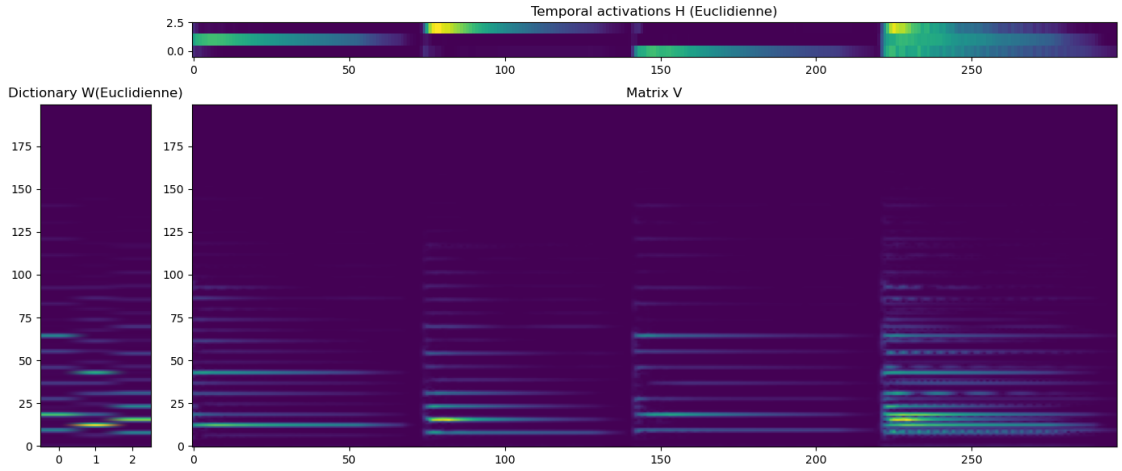
# Matrice H (Activations)
```

```
ax_H = fig.add_subplot(gs[0, 1:])
ax_H.imshow(He, aspect='auto', origin='lower')
ax_H.set_title("Temporal activations H (Euclidienne)")

# Matrice W (Dictionnaire)
ax_W = fig.add_subplot(gs[1, 0])
ax_W.imshow(We[:200], aspect='auto', origin='lower')
ax_W.set_title("Dictionary W(Euclidienne)")

# Matrice V (Originale)
ax_V = fig.add_subplot(gs[1, 1:])
ax_V.imshow(np.abs(M)[:200], aspect='auto', origin='lower')
ax_V.set_title("Matrix V")

plt.tight_layout()
plt.show()
```



[114]:
```
# Exécution KL (M doit être défini au préalable par la STFT)
Wk, Hk, costs_k = nmf_kl(np.abs(M), S=3)

fig = plt.figure(figsize=(14, 6))
gs = fig.add_gridspec(2, 3, width_ratios=[1, 4, 4], height_ratios=[1, 10])

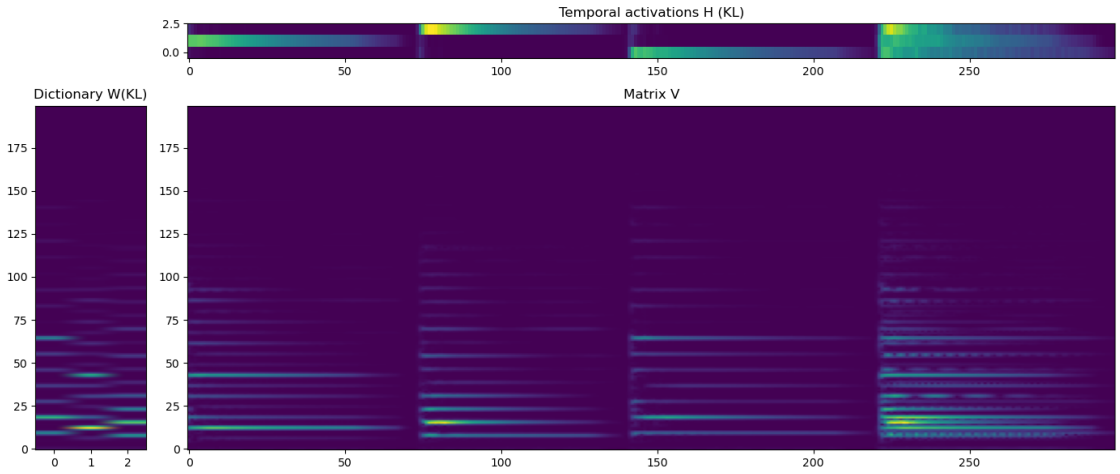
# Matrice H (Activations)
ax_H = fig.add_subplot(gs[0, 1:])
ax_H.imshow(Hk, aspect='auto', origin='lower')
ax_H.set_title("Temporal activations H (KL)")

# Matrice W (Dictionnaire)
ax_W = fig.add_subplot(gs[1, 0])
```

```
ax_W.imshow(Wk[:200], aspect='auto', origin='lower')
ax_W.set_title("Dictionary W(KL)")

# Matrice V (Originale)
ax_V = fig.add_subplot(gs[1, 1:])
ax_V.imshow(np.abs(M)[:200], aspect='auto', origin='lower')
ax_V.set_title("Matrix V")

plt.tight_layout()
plt.show()
```



[115]:
```
# Exécution IS (M doit être défini au préalable par la STFT)
Wi, Hi, costs_is = nmf_is(np.abs(M), K=3)

fig = plt.figure(figsize=(14, 6))
gs = fig.add_gridspec(2, 3, width_ratios=[1, 4, 4], height_ratios=[1, 10])

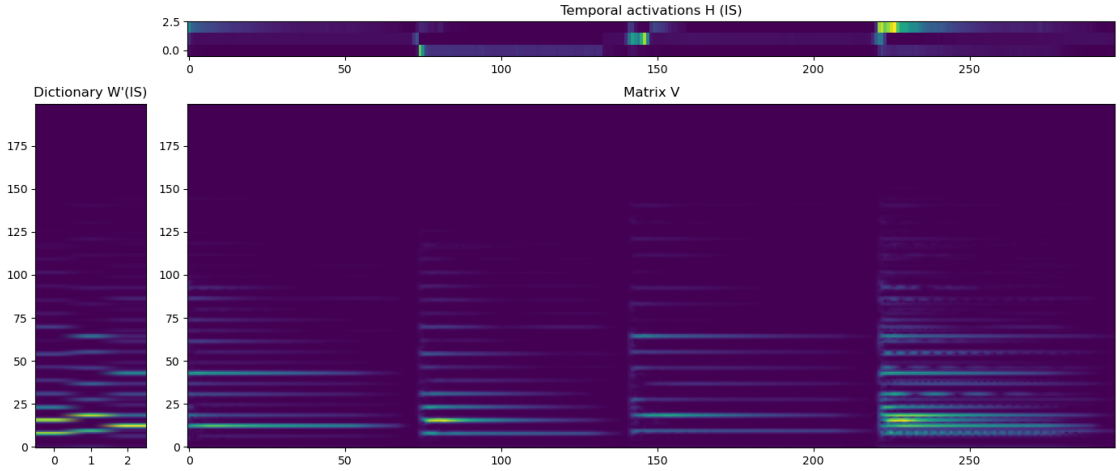
# Matrice H (Activations)
ax_H = fig.add_subplot(gs[0, 1:])
ax_H.imshow(Hi, aspect='auto', origin='lower')
ax_H.set_title("Temporal activations H (IS)")

# Matrice W (Dictionnaire)
ax_W = fig.add_subplot(gs[1, 0])
ax_W.imshow(Wi[:200], aspect='auto', origin='lower')
ax_W.set_title("Dictionary W'(IS)")

# Matrice V (Originale)
ax_V = fig.add_subplot(gs[1, 1:])
ax_V.imshow(np.abs(M)[:200], aspect='auto', origin='lower')
```

```
ax_V.set_title("Matrix V")

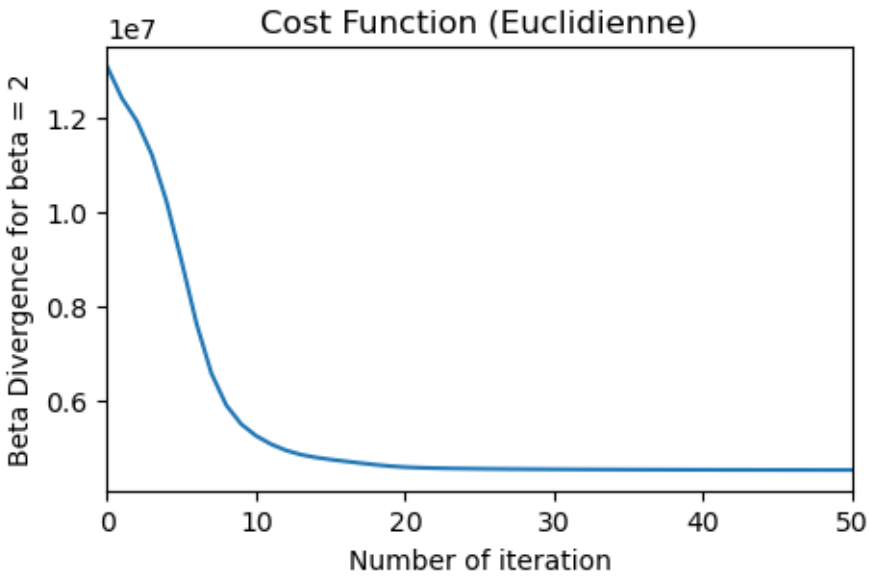
plt.tight_layout()
plt.show()
```



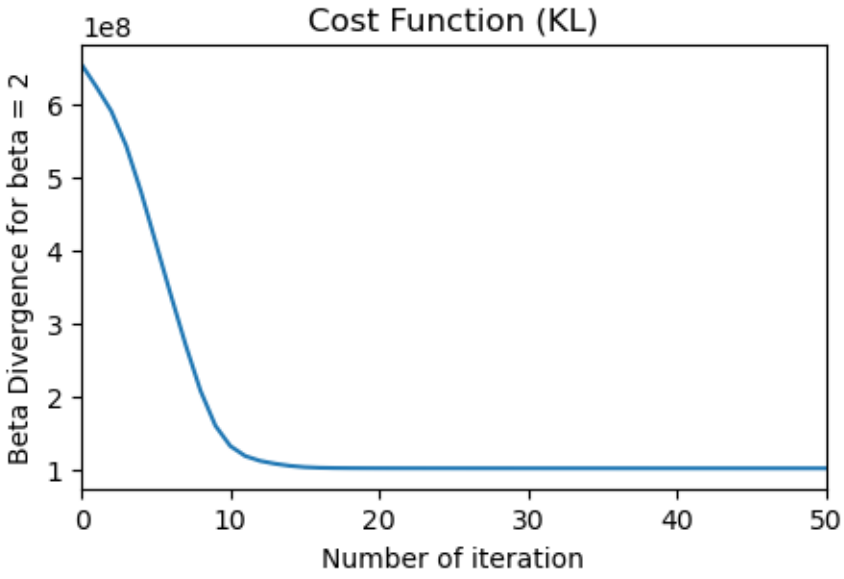
2.2.10 2.2.6 Visualisation de la convergence

Le graphique ci-dessous illustre la décroissance de la fonction de coût au cours des itérations.

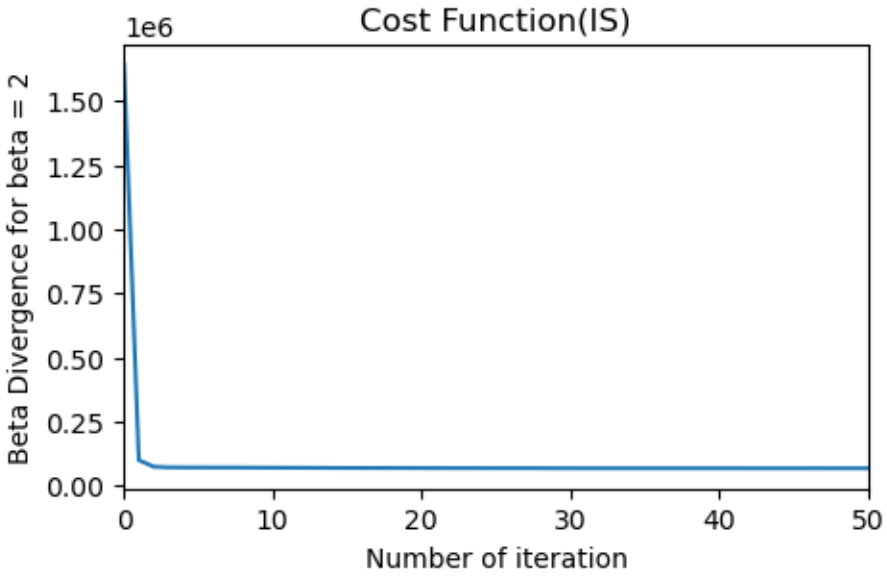
[71]:
```
plt.figure(figsize=(5,3))
plt.plot(costs_E)
plt.title("Cost Function (Euclidienne)") #
plt.xlabel("Number of iteration") #
plt.ylabel("Beta Divergence for beta = 2") #
plt.xlim(0, 50) #
plt.show()
```



[72]:
```
plt.figure(figsize=(5,3))
plt.plot(costs_k)
plt.title("Cost Function (KL)") #
plt.xlabel("Number of iteration") #
plt.ylabel("Beta Divergence for beta = 2") #
plt.xlim(0, 50) #
plt.show()
```



[73]:
```
plt.figure(figsize=(5,3))
plt.plot(costs_is)
plt.title("Cost Function(IS)") #
plt.xlabel("Number of iteration") #
plt.ylabel("Beta Divergence for beta = 2") #
plt.xlim(0, 50) #
plt.show()
```



2.2.11 Observations

Pourquoi les matrices IS semblent-elles "floues" ?

Contrairement à la méthode Euclidienne, les résultats avec Itakura-Saito paraissent plus diffus pour deux raisons : 1. **Sensibilité aux faibles énergies** : IS est invariante à l'échelle et accorde autant d'importance aux sons faibles qu'aux sons forts, conservant ainsi le bruit de fond et les harmoniques faibles. 2. **Capture de la résonance** : Le flou représente la réverbération naturelle de l'instrument. L'algorithme respecte la physique du son en ne coupant pas la note brutalement.

3 3. Reconstruction des sources sonores

[33]:
```
#Paramètre de base
fs, x = read('piano_mix.wav')
t_x = np.arange(len(x))/fs

nperseg = 2048
hop_fraction = 0.25  # 75% overlap
window = 'hann'
```

```
hop = int(nperseg * hop_fraction)
noverlap = nperseg - hop

M, freqs, times, meta = stft(
    x, fs, nperseg,
    window=window,
    hop_fraction=hop_fraction,
    output="complex",
)

seuil = 0.10
nb_source = 3
sample_freq = fs
signal = x
D, A, costs_k = NMF_Euclidienne(np.abs(M), S=3)
file_output = "source"
```

[34]:
```
sources = []
t_sources = []
```

3.1 3.1 Principe général de la reconstruction

Après la décomposition de la matrice obtenue par la STFT par factorisation NMF, le signal est représenté sous la forme :

$$M \approx D \times A$$

où: - M est le spectrogramme du signal d'entrée, (matrice obtenue par la STFT), - D est la matrice Dictionnaire, contenant les signatures fréquentielles des sources, - A est la matrice d'Activation temporelle associé à chaque source

L'objective de la reconstruction est de repasser du domaine temps-fréquence au domaine temporel et ainsi d'obtenir des fichiers audio exploitable

3.2 3.2 Reconstruction par source

Pour chaque source i, on reconstruit une matrice partielle M_i en calculant le produit extérieur entre la i-ème colonne de D et la i-ème ligne de A.

$$M_i = D_{:,i} \times A_{i,:}$$

Afin de limiter l'influence des composantes de faible amplitude, un seuil est appliqué. Par exemple, les coefficients inférieurs à 10% de la valeur maximale sont mis à zéro. Cette opération permet de réduire le bruit et donc d'améliorer la qualité (graphiquement ou audio).

$$M_i(f,t) = \begin{cases} M_i(f,t) & \text{si } M_i(f,t) > 0.10 \cdot \max(M_i) \\ 0 & \text{sinon} \end{cases}$$

[90]:
```
Mis = []
for i in range(nb_source):
    Mi = np.outer(D[:,i], A[i,:])
    maxMi = np.max(np.abs(Mi))
    Mi = np.where(Mi > maxMi * seuil, Mi, 0)

    Mis.append(Mi)
```

3.3 3.3 Reconstruction par ISTFT

Nous avons appliqué la factorisation NMF sur le module du spectrogramme, ainsi on conserve la phase d'origine du spectrogramme, ce qui permet une meilleure reconstruction. Pour chaque source, le spectrogramme reconstruit (module) est combiné avec la phase du spectrogramme original :

$$Z_i = M_i \cdot e^{j\phi(M_i)}$$

On applique ensuite l'ISTFT de scipy à ce spectrogramme complexe afin d'obtenir le signal temporel associé à chaque source. Les paramètres sont identiques à ceux donnés à la STFT afin de garentir une reconstruction fidèle (même temporalité).

[91]:
```
Zis = []
phase = np.angle(M)
for i in range(nb_source):
    Zi = Mis[i] * np.exp(1j * phase)

    Zis.append(Zi)
```

[92]:
```
for i in range(nb_source):
    t_source, source = scipy_istft(Zis[i], fs=sample_freq, nperseg=nperseg,
                                   noverlap=noverlap, window=window)
    sources.append(source)
    t_sources.append(t_source)
```

3.4 3.4 Normalisation et sauvegarde

Une fois le signal reconstruit, une normalisation est effectuée afin de conserver des amplitudes comparable au signal d'origine. Les signaux sont ensuite convertis en format entier sur 16 bits afin d'être sauvegardé sous la forme de fichoer .wav

[93]:
```
for i in range(nb_source):
    sources[i] = sources[i] * np.max(np.abs(signal)) / np.max(np.
 ↪abs(sources[i]))
    sources[i] = sources[i].astype(np.int16)

    output = f"{file_output}_{i}.wav"
    write(output, rate=sample_freq, data=sources[i])
```

[94]:
```
#Fonction regroupant toute les manipulations

def reconstruction(M, A, D, signal, t_signal, sample_freq,
                   nb_source, nperseg, hop_fraction, window,
                   file_output_name, seuil):

    hop = int(nperseg * hop_fraction)
    noverlap = nperseg - hop

    sources = []
    t_sources = []
    for i in range(nb_source):
        m = np.outer(D[:,i], A[i,:])
        maxM = np.max(np.abs(m))
        m = np.where(m > maxM * seuil, m, 0)

        z = m * np.exp(1j * np.angle(M))
        t_source, source = scipy_istft(z, fs=sample_freq, nperseg=nperseg,
                                       noverlap=noverlap, window=window)

        source = source * np.max(np.abs(x)) / np.max(np.abs(source))
        source = source.astype(np.int16)

        sources.append(source)
        t_sources.append(t_source)

        output = f"{file_output_name}_{i}.wav"
        write(output, rate=sample_freq, data=source)

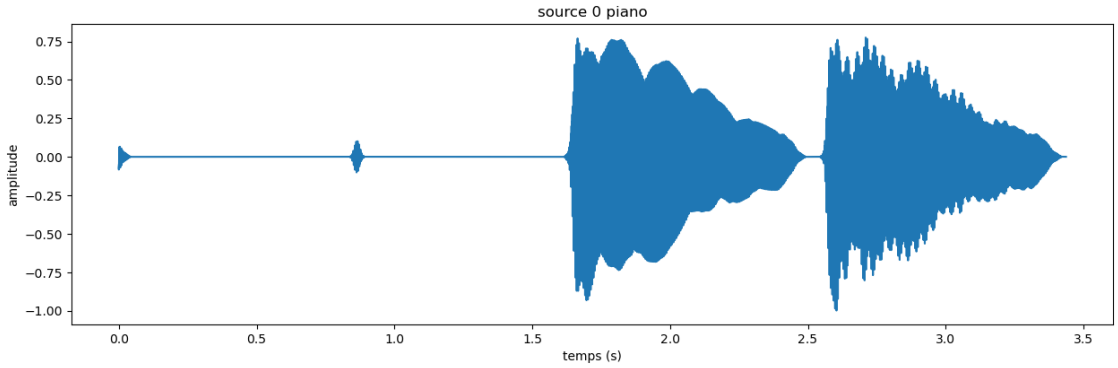
    return M, A, D, sources, t_sources
```

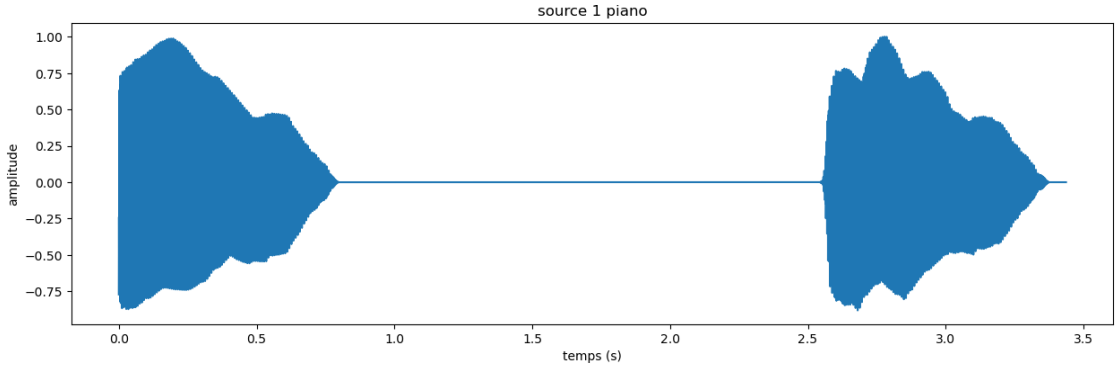
[95]:
```
M, A, D, sources, t_sources = reconstruction(M,A,D,signal, t_x, sample_freq,
                                             nb_source, nperseg, hop_fraction,
                                             window, file_output, 0.10)
```

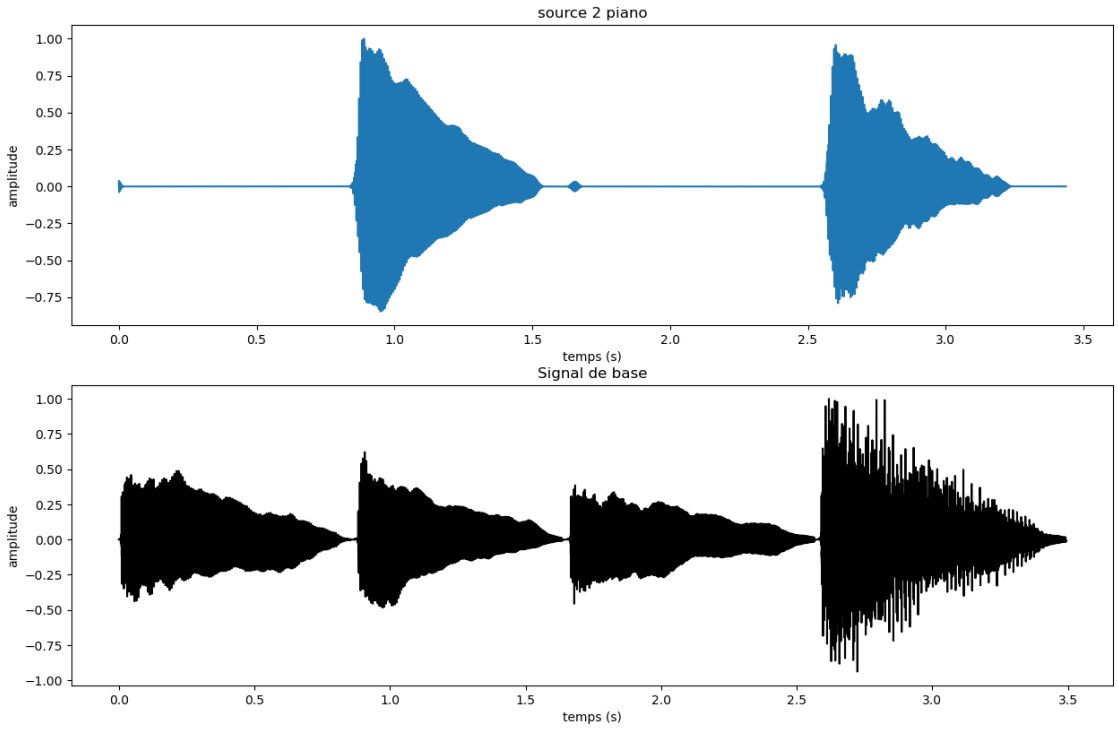
[96]:
```
for i in range(nb_source):
    plt.figure(figsize = (15,20))
    plt.tight_layout()
    plt.subplot(nb_source + 1 , 1 , i+1)
    plt.plot(t_sources[i],sources[i]/np.max(np.abs(sources[i])))
    plt.title(f'source {i} piano')
    plt.xlabel('temps (s)')
    plt.ylabel('amplitude')

plt.subplot(nb_source + 1,1,nb_source + 1)
plt.plot(t_x, x/np.max(np.abs(x)), color = 'black')
plt.title('Signal de base')
```

```
plt.xlabel('temps (s)')
plt.ylabel('amplitude');
```







4 4. Limites et amélioration

Plusieurs piste d'amélioration existent. Tout d'abord, le choix de sources constitue un paramètre important pour la NMF. Dans ce projet nous l'avons fixé, mais une estimation automatique du nombre optimal de ressource permettrait d'adapter la décomposition rapidement à n'importe quelle signal.

Ensuite, les choix des paramètres de la STFT implique aussi fortement sa qualité. En particulier la taille de la fenêtre (nperseg). Une étude plus approfondie visant à déterminer une valeur optimale de ce paramètre pourrait améliorer les performances du systèmes.

D'autre part, la reconstruction temporelle repose sur la ISTFT de scipy. Il serait mieux de l'implémenter nous-même afin d'avoir la main dessus.

Une autre amélioration possible serait l'implémentation de la Synchrosqueezing Transform (SST) qui permet une meilleure concentration de l'énergie fréquentielle et ainsi être plus précis.

Enfin, bien que ce projet ne concerne que les signaux audio, la méthodologie employée pourrait être étendue à des types de données différents comme des images ou des vidéos.

5 5. Conclusion

Ce projet avait pour objectif la mise en oeuvre de séparation de sources à partir de plusieurs outils issus du traitement du signal et de l'analyse numérique. La STFT a permis de représenter le signal dans le domaine temps-fréquence, ainsi permettant l'application de décomposition matricielle.

La factorisation NMF nous a permis de décomposer le spectrogramme du signal en matrice dictionnaire des sources et activation temporelle associées. Cette décomposition a permis directement la séparation des sources et a rendu possible la reconstruction.

La reconstruction a permis de repasser dans le domaine temporel, en combinant les spectrogrammes reconstruits (module) avec la phase d'origine et en utilisant l'ISTFT.

Les résultats obtenus nous montrent que la qualité dépend de nombreux paramètres comme le nombre de sources, les paramètres de la STFT et le seuil appliqué.

Enfin, ce projet permet d'envisager de nombreuses améliorations, que nous avons citées auparavant.